

Python — Functions & Return Values

Grade 3 | Coding



What is a Function?

A function is a reusable block of code.

Think of it like a machine:

1. You give it something (input)
2. It does a job
3. It gives something back (output)

Functions help us avoid repeating code!



Real-Life Example

Imagine a juice machine:

1. You put in oranges (input)
2. The machine squeezes them (process)
3. You get orange juice (output)

A function works the same way!

Input -> Process -> Output



The def Keyword

We use def to create a function:

```
def say_hello():  
    print("Hello, friend!")
```

- def means 'define'
- say_hello is the function name
- The colon : starts the function body
- Indented code belongs to the function



Calling a Function

Creating a function doesn't run it!
You must call it by name:

```
def say_hello():  
    print("Hello, friend!")
```

```
say_hello() # This runs it!
```

Output: Hello, friend!



Try It Yourself!

Type this in Python:

```
def greet():  
    print("Namaste!")  
    print("Welcome to coding!")
```

```
greet()
```

What do you see on screen?



Parameters (Inputs)

Parameters let us send information into a function:

```
def greet(name):  
    print("Hello, " + name + "!")
```

- name is a parameter
- It's like a blank space the function fills in
- We give it a value when we call it



Example with a Parameter

```
def greet(name):  
    print("Hello, " + name + "!!")
```

```
greet("Aarav")  
greet("Priya")
```

Output:
Hello, Aarav!
Hello, Priya!

Same function, different inputs!



Multiple Parameters

Functions can take more than one input:

```
def add(a, b):  
    print(a + b)
```

```
add(3, 5)    # Output: 8  
add(10, 20)  # Output: 30
```

Separate parameters with commas!



What is return?

return sends a value BACK from the function:

```
def add(a, b):  
    return a + b
```

- print() shows on screen
- return gives the answer back to your code
- You can save it in a variable!



Return Example

```
def add(a, b):  
    return a + b
```

```
result = add(4, 6)  
print(result)
```

Output: 10

The function gave back 10,
and we stored it in result!



Using Returned Values

```
def double(n):  
    return n * 2
```

```
x = double(7)  
print(x)      # Output: 14  
print(double(3)) # Output: 6
```

You can use return values
anywhere in your code!



Practice 1

Write a function called square that takes a number and returns it squared:

```
def square(n):  
    return n * n
```

```
print(square(5)) # What prints?  
print(square(9)) # What prints?
```



Practice 2

Write a function called `is_even` that returns `True` if a number is even:

```
def is_even(n):  
    return n % 2 == 0
```

```
print(is_even(4)) # True  
print(is_even(7)) # False
```

Hint: `%` gives the remainder!



Key Takeaways

- def creates a function
- Call it by name with ()
- Parameters are inputs
- return sends a value back
- Functions make code reusable & neat!

Functions = your coding superpower!



Great Job!

You learned how to:

- Create functions with def
- Add parameters for inputs
- Use return to get values back

Keep practising — you are becoming a Python pro!

